

ABC Company

Development Process & Methodology

Document ID	ABC-DOC-007
Version	v1.0
Date	June 2026
Prepared By	ABC Company — Engineering
Classification	Business Confidential

ABC Company — Colombo | London | Melbourne

Table of Contents

- 1. Agile Framework Overview
- 2. Project Lifecycle — Six Phases
- 3. Quality Standards
- 4. Tools and Tooling

1. Agile Framework Overview

ABC Company's engineering delivery is grounded in Agile principles, adapting the most appropriate framework to the nature and phase of each client engagement. Rather than applying a rigid one-size-fits-all process, the company draws primarily on Scrum for active feature development, Kanban for ongoing maintenance engagements, and a hybrid Scrumban approach for projects that transition from intensive development into steady-state operations. This pragmatic stance allows ABC Company to optimise its process for the specific dynamics of each project — the team's size, the client's availability, the nature of the work, and the stage of the product lifecycle — without abandoning the core Agile values of iterative delivery, continuous improvement, and close client collaboration.

Scrum is ABC Company's default methodology for new product development and feature-intensive engagements. Sprints run for two weeks, a cadence the company has found to strike the optimal balance between maintaining development momentum and providing clients with frequent, tangible progress checkpoints. Each sprint follows a structured cycle of four ceremonies, each designed to create alignment, surface impediments early, and continuously improve the team's process.

Sprint Planning takes place on the first day of each sprint and is timeboxed to two hours. During planning, the Scrum Master or Project Manager facilitates a collaborative session in which the Product Owner or client representative presents the prioritised backlog items to the development team. Stories are estimated using story points on the Fibonacci scale (1, 2, 3, 5, 8, 13), which deliberately captures the non-linear growth in complexity and uncertainty as tasks grow larger. The team collectively commits to a sprint goal — a concise statement of what the sprint will deliver in terms of user value — and selects the backlog items that will comprise that sprint's scope. Participants always include the Scrum Master or Project Manager, the full development team, and the Product Owner or a nominated client representative. Sprint planning is the moment where the team's velocity, the client's priorities, and technical constraints are brought into alignment before a single line of code is written.

The Daily Standup runs every morning for no more than fifteen minutes. Each team member answers three questions: what did they accomplish yesterday, what will they work on today, and are there any blockers preventing progress. The standup is intentionally brief — it is a coordination event, not a status report — and the Scrum Master is responsible for keeping it focused and ensuring that any blockers identified are resolved through follow-up conversations outside the standup itself. For geographically distributed teams with challenging time zone overlaps, ABC Company offers an asynchronous standup option delivered via a Slack bot: team members post their three answers before a designated cut-off time each morning, and the Project Manager reviews and flags anything requiring immediate attention. This async option preserves the information-sharing benefits of the standup without requiring engineers in distant time zones to join calls at unsociable hours.

The Sprint Review and Demo takes place on the last day of the sprint, is timeboxed to forty-five minutes, and is the primary client-facing ceremony of the sprint cycle. The development team demonstrates the completed work to client stakeholders, walking through each delivered story against its acceptance criteria. Stakeholders ask questions, provide feedback, and formally accept or reject stories based on whether the acceptance criteria have been met. This ceremony is deliberately not a passive presentation — it is a collaborative review that generates the insights and feedback that refine the backlog for the next sprint. Clients with limited availability are particularly encouraged to attend this ceremony, as it is the most efficient use of their time for maintaining alignment with the engineering team's progress.

The Sprint Retrospective follows the demo and is limited to thirty minutes. It is an internal team ceremony focused on continuous process improvement rather than product features. The team reflects on three questions: what went well in the sprint, what could be improved, and what one concrete action will the team commit to in the next sprint to address the most significant improvement opportunity. ABC Company tracks retrospective action items across sprints, monitoring improvement velocity over time and ensuring that commitments made in retrospectives are actually

implemented rather than forgotten in the next sprint's urgency. This discipline — treating the process itself as a product that deserves iteration — is a significant factor in the steady improvement in team throughput and quality that clients observe over the life of longer engagements.

Kanban, Scrumban, and the Definition of Done

For ongoing support and maintenance engagements, ABC Company uses Kanban boards with work-in-progress (WIP) limits in place of the sprint cadence. Kanban suits maintenance work because incoming requests are inherently unpredictable in timing and scope — bug reports, infrastructure changes, and minor feature requests arrive on a continuous basis rather than in predictable batches that can be planned two weeks in advance. The Kanban board is organised into swimlanes for Bug Fixes, Feature Requests, Infrastructure, and Technical Debt, giving the client and the team clear visibility into what work is in progress and where bottlenecks are forming. WIP limits prevent the team from taking on more concurrent work than it can complete efficiently, a discipline that reduces context switching, shortens cycle times, and produces more reliable delivery of individual items.

Some projects begin with a Scrum cadence during the active product-building phase and naturally transition to Kanban as the product matures and moves into a maintenance and incremental improvement mode. ABC Company calls this the Scrumban approach: the team retains the sprint structure for planned feature work while treating incoming maintenance items as a continuous-flow Kanban stream running in parallel. Clients benefit from the predictability of sprint-based feature delivery and the responsiveness of Kanban for unplanned operational requests, without the friction of managing two completely separate engagements or engineering teams.

Regardless of the framework in use, ABC Company maintains a rigorous Definition of Done that must be satisfied before any story is considered complete. A story is done when the code has been reviewed and approved via pull request, all automated tests pass, the feature is covered by tests meeting the project's coverage threshold, inline documentation has been updated where required, and the change has been successfully deployed to the staging environment. Stories that do not meet the Definition of Done are not demonstrated in the Sprint Review and are not counted in the sprint's velocity. This standard is non-negotiable: it exists to prevent the accumulation of 'almost done' work that quietly accumulates technical debt and erodes team confidence in the codebase over time.

Velocity — the number of story points the team completes per sprint — is tracked from the first sprint and used for forecasting after the team's velocity stabilises, typically by sprint three or four. Clients receive a weekly burndown chart showing story points completed versus remaining within the current sprint, providing a simple and honest signal of whether the sprint goal is on track. Burndown data is shared without manipulation: if a sprint is running behind, the client knows early enough to make informed decisions about scope, and the team has the opportunity to address the causes rather than simply working harder toward an unrealistic deadline.

2. Project Lifecycle — Six Phases

ABC Company structures all client engagements around a six-phase project lifecycle that covers the full journey from initial discovery to long-term support. The phases are sequential in their primary outputs but overlap in practice — testing begins during development, design artefacts are refined in response to development discoveries, and support planning begins before launch. Each phase has defined entry criteria, deliverables, and a client review gate before the next phase commences.

Phase 1: Discovery

Typical Duration

Duration: 1 to 3 weeks

The Discovery phase is the foundation on which all subsequent work rests. Its purpose is to establish a shared understanding — between ABC Company and the client — of the business goals, user needs, technical constraints, and success criteria that will govern the engagement. A Discovery phase conducted poorly produces an expensive fiction: a requirements document that everyone has signed but that does not reflect what the business actually needs. ABC Company invests significant effort in doing Discovery rigorously precisely because it is the cheapest and most effective form of risk management available.

Stakeholder interviews form the backbone of Discovery. ABC Company conducts structured one-on-one and group interviews with all key stakeholders: product owners, end users, operations teams, technical leads, and executive sponsors. Each interview is designed to surface not just stated requirements but underlying motivations, constraints, and concerns that are often more important than the initial feature list. The output of these interviews is a stakeholder map identifying the decision-making hierarchy, a goals hierarchy connecting business objectives to product features, and a set of measurable success metrics that will be used to evaluate the engagement's outcomes. Clients commit to providing three to five hours per week of stakeholder availability during the Discovery phase — this is a non-negotiable input, because no amount of engineering skill can substitute for direct access to the people who understand the problem domain.

Requirements gathering during Discovery produces user stories with clearly defined acceptance criteria, use case diagrams for complex multi-actor workflows, and a comprehensive set of non-functional requirements specifying performance, security, scalability, availability, and accessibility targets. These non-functional requirements are not afterthoughts — they directly influence architecture decisions and cost estimates, and establishing them in Discovery prevents the expensive retrofitting that occurs when, for example, a system designed for one hundred concurrent users turns out to need to support ten thousand.

Technical assessment during Discovery examines the client's existing systems, APIs, data sources, and infrastructure. ABC Company documents all integration points, evaluates the quality and reliability of existing components that the new system will depend on, and produces an initial risk register identifying the technical assumptions and dependencies that carry the most uncertainty. Competitive analysis — reviewing similar products and solutions in the market — rounds out the Discovery work, helping the team identify opportunity areas and avoid patterns that are known to be problematic in the domain.

Discovery deliverables include a project charter, a requirements document, an initial architecture proposal, a risk register with identified mitigations, and a project timeline with key milestones. These documents are reviewed with the client in a structured Discovery Readout session before Phase 2 begins. The readout is a decision gate: if the Discovery outputs reveal that the project is significantly more complex, more expensive, or less commercially viable than the initial brief suggested, both parties have the opportunity to adjust scope, timeline, or budget before substantial development resources are committed.

Phase 2: Design

Typical Duration

Duration: 2 to 4 weeks

The Design phase translates the requirements and architecture proposals from Discovery into detailed, validated blueprints — both visual and technical — that the development team can build from with confidence. ABC Company treats design as a distinct and critically important phase, not as a prelude to the 'real work' of development. The cost of discovering a fundamental UX or architecture flaw during design is a day of iteration in Figma; the cost of discovering the same flaw after four weeks of development is measured in re-work, team morale, and client trust.

User research during Design builds on the stakeholder interviews conducted in Discovery, developing detailed user personas that represent the primary and secondary audiences for the product. For each persona, the team constructs user journey maps tracing the end-to-end experience of completing key tasks, identifying moments of friction, uncertainty, or cognitive load that the interface design should address. Information architecture is defined — the structural hierarchy of content and functionality — before any visual design begins, ensuring that navigation and organisation are grounded in user mental models rather than the internal logic of the development team. For B2B products serving multiple user types, separate journey maps are developed for administrator and end-user flows, which frequently have very different usage patterns and interface requirements.

Wireframing produces low-fidelity representations of all key screens and interaction flows in Figma. These wireframes deliberately avoid colour, typography, and visual polish, keeping the focus on layout, information hierarchy, and interaction logic. Client review and iteration on wireframes typically requires two to three rounds before the layouts are sufficiently validated to proceed to high-fidelity visual design. This iteration is normal and expected — its purpose is to surface disagreements and assumptions early when changes are cheap, not to generate extra revision cycles.

High-fidelity design applies the client's brand guidelines — or, for new brands, a brand identity developed in collaboration with the client — to the validated wireframe layouts, producing polished visual designs and interactive Figma prototypes suitable for user testing. The Design phase also produces the foundation of a design system: a component library of reusable UI elements with documented behaviour, states, and interaction patterns. This design system becomes a shared contract between the design and development teams, ensuring visual and behavioural consistency across the product and dramatically reducing the time required to design and implement future features.

Architecture design during Phase 2 produces the technical blueprints that govern Phase 3 development. These include a system architecture diagram showing all services, data stores, and external integrations; a database schema design; an API contract defined in OpenAPI/Swagger specification format before any implementation begins; an infrastructure diagram showing cloud resource topology, network boundaries, and deployment targets; and a deployment strategy document. Security design is conducted in parallel, establishing the authentication and authorisation model, data encryption strategy at rest and in transit, and a mapping of the product's data flows to applicable compliance requirements such as GDPR, HIPAA, or PCI-DSS.

Phase 2 deliverables comprise Figma designs with interactive prototypes, an architecture design document, the OpenAPI specification, the database schema, and the infrastructure plan. These documents are owned by the client from the moment of creation and form the reference baseline for all development work in Phase 3.

Phase 3: Development

Typical Duration	Duration: 2 to 6 months (variable by scope)
------------------	---

Phase 3 is where the designs, architecture plans, and API specifications from Phases 1 and 2 are realised as working software. Development is conducted in two-week Agile sprints as described in Section 1, with stories drawn from the prioritised backlog and implemented in order of business value. The phase begins with Sprint 0, a short setup sprint dedicated to infrastructure rather than features.

Sprint 0 tasks include configuring the development, staging, and production environments; establishing the CI/CD pipeline; generating boilerplate code from the agreed architecture; running initial database migrations to create the schema defined in Phase 2; configuring linting, formatting, and type checking tools; and ensuring that every developer on the team can run the full application stack locally. The significance of Sprint 0 is frequently underestimated: a CI/CD pipeline established on day one pays compounding dividends throughout the project by catching integration failures early and enabling reliable, repeatable deployments that the team trusts.

Feature development follows Sprint 0, with stories implemented in priority order as determined by the client and the product backlog. Complex or high-risk features are subject to pair programming — two engineers working together on the same piece of code — which improves quality, accelerates knowledge transfer, and produces more robust solutions than individual development for the most technically demanding work. Every pull request requires at minimum one code reviewer; pull requests on critical paths — authentication systems, payment integrations, core data models — require two reviewers. Code reviews are not optional and are not bypassed under sprint pressure: the Definition of Done requires a merged and approved pull request.

ABC Company defaults to either GitFlow or trunk-based development depending on client preference and team size. Feature branches are short-lived — engineers are encouraged to integrate frequently rather than developing on long-running branches that diverge significantly from the main line. Automated checks run on every push: linting, type checking, and the unit test suite must all pass before a pull request can be merged. Feature flags are used for gradual rollout of large or incomplete features, allowing code to be merged to main and deployed to staging without being visible to end users until the feature is ready for release.

Technical debt is managed proactively rather than allowed to accumulate silently. Twenty percent of each sprint's capacity is allocated to technical debt reduction tasks, which are maintained in the backlog alongside feature stories and prioritised based on their impact on development velocity and system health. This allocation is transparent to clients: they can see exactly what technical debt work is being done and why, and they participate in the prioritisation conversation. The result is a codebase that remains maintainable and extensible throughout the development phase, rather than accumulating a 'debt cliff' that must be addressed in an expensive refactoring sprint before the product can be scaled.

Client involvement during Phase 3 follows a structured rhythm. Sprint demos every two weeks provide the primary progress checkpoint. Clients receive continuous access to the staging environment for ad-hoc testing between demos. A dedicated Slack channel provides a low-friction channel for asynchronous questions, feedback, and clarifications, with ABC Company committing to respond to client messages within four business hours during normal working days.

Phase 4: Testing

Typical Duration	Duration: 2 to 4 weeks, overlapping with development
------------------	--

Testing at ABC Company is not a phase that happens after development finishes — it is a practice that runs continuously in parallel with development, intensifying in the final weeks before launch. The company's testing strategy follows the test pyramid model: a broad base of unit tests, a middle layer of integration tests, and a smaller set of end-to-end tests that exercise the full application stack. The target code coverage threshold is 80 percent, with higher thresholds applied to critical business logic paths.

Unit tests form the base of the pyramid and constitute approximately 60 percent of the test suite by count. They test individual functions, classes, and modules in isolation, run in milliseconds, and provide the fastest feedback loop for developers. Integration tests, representing approximately 25 percent of the suite, test the interactions between components — service layer to database, API client to external service — at a level of fidelity that unit tests cannot provide. End-to-end tests, representing the remaining 15 percent, simulate real user interactions against a running instance of the application, verifying that complete user journeys behave as expected. All unit tests run on every commit; integration tests run on every pull request; the end-to-end suite runs nightly and before every release candidate.

Manual QA complements the automated test suite with exploratory testing — a structured but unscripted approach in which experienced QA engineers probe the application for unexpected behaviour, edge cases, and usability issues that automated tests cannot detect. Cross-browser testing covers Chrome, Firefox, Safari, and Edge; cross-device testing for mobile applications covers iOS and Android across a representative range of screen sizes and OS versions. Accessibility testing against WCAG 2.1 AA is conducted for applications with public-facing interfaces.

Performance testing uses k6, an open-source load testing tool, to simulate the expected traffic patterns against the application's API endpoints and key user flows. Load tests are designed against the non-functional requirements established in Discovery — if the product must support 5,000 concurrent users with a 95th-percentile response time under 500 milliseconds, the load test is designed to verify exactly that. Performance bottlenecks identified during load testing are addressed before the launch timeline, not after. For applications with strict SLA requirements, stress tests are run to understand the system's failure modes and ensure graceful degradation under load beyond the expected operating envelope.

Security testing runs on multiple levels. OWASP ZAP automated scanning is run against every release candidate to check for common web application vulnerabilities. Dependency vulnerability scanning is conducted continuously using Snyk, which monitors the project's dependency graph and alerts the team to newly disclosed vulnerabilities in third-party packages. For applications handling sensitive data in regulated sectors — financial services, healthcare, government — ABC Company conducts or commissions manual penetration testing by qualified security professionals, producing findings that are triaged, remediated, and documented before the application enters production.

User Acceptance Testing (UAT) gives the client's own team two weeks in the staging environment to validate the application against real-world usage scenarios before launch. ABC Company prepares structured test scripts covering all critical user journeys, but encourages the client's team to conduct additional exploratory testing beyond the scripts. All bugs and feedback collected during UAT are logged, triaged by severity, and addressed according to a prioritisation agreed with the client. Regression testing — running the full automated test suite — is performed after every bug fix during UAT to verify that fixes do not introduce regressions in previously working functionality.

Phase 5: Launch

Typical Duration	Duration: 1 to 2 weeks
------------------	------------------------

The Launch phase is the structured transition from the development and testing environment to production, executed in a manner that minimises risk, ensures observability, and provides clear rollback options in the event of unexpected issues. A chaotic or under-prepared launch is one of the most damaging events in a product's lifecycle — it erodes client confidence and can cause customer-facing incidents that take weeks to recover from in terms of user trust. ABC Company treats launch readiness as a measurable, binary state, not a subjective judgment.

Before any production deployment is made, ABC Company works through a comprehensive pre-launch checklist. Performance benchmarks must be met against the agreed SLA targets. Security scans must be clean — no outstanding critical or high vulnerabilities. Monitoring and alerting must be configured, tested, and operational before traffic is live. The backup strategy must be verified, including a tested restore procedure. A documented rollback plan must exist, specifying the exact steps and decision criteria for reverting to the previous version. DNS and SSL configuration must be complete and validated. Analytics and error tracking must be confirmed as capturing data correctly in the production environment. No item on the checklist can be deferred to post-launch: the checklist is a gate, not a suggestion.

ABC Company's standard deployment strategy for production releases is blue-green deployment, which maintains two identical production environments and routes traffic to the new version only after it has been fully deployed and health-checked. If the new version passes health checks, traffic is switched instantly with zero downtime. If the health check fails, traffic remains on the old version automatically, eliminating the risk of a degraded production environment sitting between a failed forward deployment and a manual rollback. For high-risk changes affecting core functionality, canary deployments are used: the new version initially receives ten percent of production traffic, is monitored for error rates and performance metrics, and progressively scaled to fifty percent and then one hundred percent over a period of hours or days, depending on confidence levels.

On launch day itself, ABC Company operates a virtual war room: the full project team is on standby throughout the day, monitoring dashboards are active on shared screens, and a dedicated communication channel is open with the client for real-time status updates. An automated smoke test suite runs immediately after deployment, verifying that the most critical user journeys are functional in the live environment before the URL is shared with end users.

The 48 hours following launch constitute an intensive monitoring period in which the team watches key metrics closely: error rates in Sentry, response time distributions and infrastructure performance in Datadog, uptime status from PagerDuty. Any anomaly that deviates from the baseline established during load testing triggers immediate investigation. This monitoring intensity is deliberately higher than the steady-state post-launch support model — the goal is to catch and address any unexpected production behaviour before it has time to affect a meaningful number of users.

Phase 6: Support & Maintenance

Typical Duration	Duration: Ongoing
------------------	-------------------

The relationship between ABC Company and its clients does not end at launch. Every project is covered by a 30-day post-launch warranty period during which any bugs discovered in functionality that was accepted during UAT are investigated and resolved at no additional cost. The warranty applies to defects in the delivered work — unexpected behaviour relative to the accepted acceptance criteria — not to new feature requests or changes in requirements that arise after launch. Response times during the warranty period are governed by severity: Critical issues (system unavailable or data loss risk) are addressed within four hours; High issues (significant functionality impaired) within twenty-four hours; Medium issues within forty-eight hours; Low issues within five business days.

Beyond the warranty period, ABC Company offers structured support retainers at three tiers. The Basic tier provides twenty hours of engineering time per month, suitable for small applications with low change velocity and minimal operational complexity. The Standard tier provides forty hours per month, covering regular maintenance tasks, minor feature additions, and proactive monitoring. The Premium tier provides eighty or more hours per month and is appropriate for active products that require ongoing feature development, performance optimisation, and dedicated on-call engineering support. The Services Catalog provides detailed descriptions of what is included in each tier and the applicable SLA commitments.

Continuous improvement is built into ABC Company's maintenance engagements through regular structured reviews. Clients receive monthly health reports summarising application performance metrics, error rates, infrastructure costs, and any notable events or incidents from the preceding month. Quarterly architecture reviews assess whether the system's design continues to serve the product's evolving needs and identify any structural changes that should be planned in the coming months. Annual security audits ensure that the application's security posture keeps pace with the evolving threat landscape and any changes in the regulatory environment.

Technology currency is maintained through a structured update cadence. Dependency updates are applied monthly, with security patches applied immediately upon disclosure regardless of the regular maintenance window. Framework major version upgrades are planned quarterly, with appropriate testing and staged rollout to minimise disruption. Infrastructure optimisation reviews identify opportunities to reduce cloud costs, improve performance, or simplify operational complexity as usage patterns mature and evolve. Clients who remain on maintenance retainers with ABC Company consistently report that their applications remain technically current and operationally healthy years after initial launch — an outcome that reflects the compounding value of proactive, structured maintenance versus reactive break-fix support.

3. Quality Standards

Quality at ABC Company is not a layer applied on top of the development process — it is woven into the process itself through standards, tools, and practices that operate continuously rather than at discrete checkpoints. The following standards apply to every project the company delivers, regardless of size, budget, or technology stack. They are not aspirational guidelines but operational requirements that the team is accountable for meeting on every engagement.

1. Code Reviews

Every pull request at ABC Company requires at least one peer review before it can be merged, regardless of the seniority of the author or the apparent triviality of the change. Pull requests touching critical paths — authentication and authorisation systems, payment processing, data migration scripts, or any code handling personally identifiable information — require a minimum of two independent reviews. Reviewers work through a consistent review checklist that evaluates correctness (does the code do what it claims to do), performance (are there obvious inefficiencies or N+1 query patterns), security (are external inputs validated, are secrets absent from the code), readability (is the code understandable by a new team member without the author present to explain it), test coverage (are the appropriate tests present and are they testing the right behaviour), and documentation (are public interfaces documented and are complex algorithms explained). Code review is not treated as a bottleneck or a formality — it is the primary quality gate in the development process and an important mechanism for knowledge transfer within the team.

2. Testing Standards

ABC Company's testing standard requires a minimum of 80 percent code coverage across all projects, measured as a hard threshold in the CI pipeline — a build that drops below the coverage threshold fails and cannot be merged. All new features must be accompanied by tests at the appropriate levels of the test pyramid before the implementing pull request can be approved. Bug fixes are subject to a specific discipline: a regression test that reproduces the bug must be written and committed alongside the fix, ensuring that the same bug cannot silently reappear in a future refactoring. No code is merged to the main branch when tests are failing — not under sprint pressure, not for hotfixes, not for 'temporary' workarounds. This standard is enforced by the CI pipeline, which acts as an objective gatekeeper that is immune to the social pressures that sometimes lead teams to bend testing standards when deadlines approach.

3. CI/CD Pipeline

Every project receives a fully configured continuous integration and continuous delivery pipeline from the first day of Sprint 0. The standard ABC Company pipeline executes the following stages in order on every push to a feature branch or pull request: linting to enforce code style rules, type checking to catch type errors before runtime, unit tests to verify component behaviour, integration tests to verify component interactions, and a build step to produce the deployment artefact. On merge to the main branch, the pipeline additionally deploys the artefact to the staging environment automatically. Deployments to production require a manual approval step — a human being must review and authorise the production release — providing a final gate against accidental or premature production deployments. The pipeline is treated as a first-class engineering asset: it is documented, monitored for reliability, and updated when new quality checks or deployment targets are added.

4. Linting and Formatting

ABC Company enforces consistent code style across all projects through automated tooling rather than style guide documents and manual enforcement. For JavaScript and TypeScript projects, ESLint is configured with a project-standard rule set and Prettier handles all formatting. For Python projects, Black is used for formatting and Ruff for linting, providing sub-second feedback that is significantly faster than the traditional Flake8/pylint combination. All linting and formatting tools are enforced via pre-commit hooks, ensuring that engineers cannot commit non-conforming code even accidentally. The CI pipeline also runs these checks independently as a safety net. The effect of this approach is that code style is never a subject of debate in code reviews — the tools decide, consistently and without subjectivity, freeing reviewers to focus on correctness, design, and security rather than indentation.

5. Security Practices

ABC Company builds security considerations into the development process at every stage rather than treating security as an audit to be passed at the end. All developers complete annual OWASP Top 10 awareness training and are expected to apply that knowledge in their daily work. Input validation is applied to all data received from external sources — user inputs, API responses, file uploads — before that data is used in application logic or persisted to a database. Parameterised queries are mandatory for all database interactions; raw SQL string concatenation is considered a blocking issue in code review. Secrets management is handled through environment variables and secret management services — no credentials, API keys, or connection strings appear in source code or version control history. HTTPS is enforced everywhere: no application developed by ABC Company is deployed to production without TLS termination. Dependency vulnerability scanning runs continuously via Snyk, and security patch updates are applied to all projects on maintenance retainers immediately upon disclosure.

6. Architecture Standards

ABC Company's architecture decisions are guided by clean architecture and hexagonal architecture principles, which separate the application's core business logic from its infrastructure concerns — databases, external APIs, message queues — through well-defined interface boundaries. This separation makes the application's core behaviour independently testable and makes it possible to swap infrastructure components without rewriting business logic. API-first design is practised on every project: the OpenAPI specification is created and agreed upon before implementation begins, ensuring that the API contract is driven by client and integration needs rather than by implementation convenience. Database schema changes are managed through versioned migration files using Alembic for Python projects or Prisma Migrate for TypeScript projects, ensuring that schema evolution is auditable, repeatable, and reversible. All cloud infrastructure is defined as code using Terraform, providing a version-controlled, peer-reviewed record of every infrastructure resource and its configuration. Feature flags are used for gradual rollout of architectural changes and risky new features, allowing the team to deploy code to production before it is fully activated and to deactivate features instantly if problems emerge without requiring a code rollback.

7. Documentation Standards

Every repository maintained by ABC Company contains a README that provides a new developer with everything they need to understand the project's purpose, set up their local development environment, run the test suite, and understand the deployment process. API documentation is auto-generated from the OpenAPI specification and published to a URL accessible to the client and all integration partners. Architecture Decision Records (ADRs) are written for every significant technical decision — the choice of database, the messaging architecture, the authentication strategy — documenting the context, the options considered, and the reasoning behind the decision made. ADRs are invaluable for onboarding new team members and for revisiting past decisions when circumstances change. Runbooks are maintained for all operational procedures — deployments, incident response, backup restoration, common debugging procedures — ensuring that operational knowledge is not locked in the heads of individual engineers. User guides are produced for all client-facing features in applications where the end users require documentation to use the system effectively.

4. Tools and Tooling

ABC Company selects tools based on proven reliability, team familiarity, and client integration requirements rather than novelty. The toolchain below represents the standard configuration used across most engagements; alternatives can be accommodated where clients have strong preferences or existing infrastructure investments that make a different tool choice more practical.

Category	Tool	Purpose
Project Management	Jira / Linear	Sprint planning, backlog management, bug tracking
Communication	Slack	Daily async communication, standup bots, client channels
Video Calls	Google Meet / Zoom	Standups, sprint demos, planning, client reviews
Design	Figma	UI/UX wireframing, high-fidelity design, interactive prototyping
Version Control	GitHub / GitLab	Code hosting, pull requests, branch protection
CI/CD	GitHub Actions / GitLab CI	Automated testing, linting, staging and production deploys
Infrastructure	Terraform	Infrastructure as Code for all cloud resources
Containers	Docker + Kubernetes	Containerisation, orchestration, and scaling
Monitoring	Datadog / Grafana	APM, metrics, dashboards, alerting
Error Tracking	Sentry	Application error monitoring and release tracking
Documentation	Confluence / Notion	Project documentation, ADRs, runbooks, guides
API Design	Swagger / Stoplight	OpenAPI specification authoring and publishing

Client Access to Project Tools

ABC Company configures client access to all relevant project management and communication tools as part of Sprint 0 setup. Clients receive named accounts in the project management tool (Jira or Linear, depending on preference), are added to the dedicated Slack workspace, and are given viewer or contributor access to the Figma project for design review. Clients who prefer to use their own Jira or project management instance can accommodate this — ABC Company will work within the client's tooling environment rather than requiring the client to adopt ABC Company's preferred stack.

Transparency is a core principle behind ABC Company's tooling policy. Clients are never locked out of the systems that govern their project. At any point during an engagement, the client can log in to the project management tool and see the full backlog, the current sprint's status, the velocity history, and the team's capacity. They can review open pull requests in the version control system, examine the CI/CD pipeline results, and access the staging environment directly. This level of access is not merely symbolic — clients who exercise this access routinely report that it gives them significantly more confidence in the engagement's progress than the periodic status reports they received from previous development partners.